Frontend Explanation and Demo Guide

Project: Rate-Limiting and Threshold-Based DDoS Mitigation in Software Defined Networking (SDN)

This guide is for presenting the project and its dashboard to a teacher. It explains what the dashboard shows, where every number comes from, gives a timed 5–8 minute demonstration script with the exact commands, and prepares you for viva questions. Everything described here exists in the repository and was tested on the project machine on 23 October 2026 (see TEST_REPORT.md).

A. Project introduction

Problem. A Distributed Denial-of-Service (DDoS) attack floods a victim with traffic from many sources until legitimate users can no longer be served. In a Software Defined Network (SDN) the switches are simple and a central **controller** decides how traffic is forwarded, using the **OpenFlow** protocol. That central control makes SDN a good place to detect and stop attacks — but the controller and the switch's flow table are also new targets.

Why it is hard. A crude flood is easy to spot. The hard case is an **adaptive low-rate attack that imitates a flash crowd**: a flash crowd is a genuine, sudden surge of real users (for example, everyone opening a results page at once). The attack copies its shape: many sources, realistic rates, natural jitter, a gradual ramp-up — and it pauses ("feints") so that detectors which count suspicion over a time window are reset. Block too eagerly and you punish real users; block too late and the attack succeeds.

Our approach. A lightweight, **non-machine-learning** pipeline:

1. **Member 1** generates four kinds of traffic in a Mininet testbed: normal, genuine flash crowd, naive low-rate DDoS, and the adaptive flash-crowd-mimicking DDoS (subtle / moderate / aggressive).
2. **Member 2** measures the network every second through OpenFlow and produces a feature vector (packet rate, flow-table size, Packet-In rate, entropy, jitter …).
3. **Member 3** classifies every second with simple statistics: a fixed **threshold**, a **CUSUM** change detector, and a **temporal** rule that recognises "burst – pause – burst" patterns.
4. **Member 4** (mitigation and evaluation) turns attack decisions into **OpenFlow meters** that cut only the offending hosts down to a normal rate, removes them when the attack ends, and measures everything.

What the frontend demonstrates. The dashboard makes the invisible visible: you can watch traffic enter the switch, see the detector's decision each second, see the moment mitigation switches on and which hosts are limited, and see that legitimate traffic still gets through — live, during a real Mininet experiment. A second view compares the proposed system with baseline configurations using recorded results.

B. Frontend technology stack

Part	Actual technology	Why
Page	One file, member4_mitigation/dashboard/index.html: HTML5, CSS, plain JavaScript (no framework)	No build step, no Node.js, works offline in a lab; nothing to install
Charts	Hand-written SVG drawing code in the same file (no chart library)	Offline, small, full control over units, shading and tooltips
Backend / API	member4_mitigation/dashboard/server.py — Python standard library http.server (ThreadingHTTPServer)	No extra packages; same language as the rest of the project
Data	state.py reads the CSV/JSON files each experiment writes; control.py starts/stops experiments (control mode only)	The files are the single source of truth; the page never invents values

Part	Actual technology	Why
Communication	Browser polls JSON over HTTP: state every 1 s, control status every 2 s, run list and health every 5 s. No WebSocket.	Simple and robust: a missed poll is just retried
SDN side	os-ken 4.2.2 (maintained fork of Ryu) controller, OpenFlow 1.3, Open vSwitch 3.7.1, Mininet 2.3.0	The project's existing testbed

Starting it:

```
python3 member4_mitigation/dashboard/server.py               # read-only (no root)
sudo python3 member4_mitigation/dashboard/server.py --control  # with Start/Stop buttons
```

URL: **http://127.0.0.1:8050/** (localhost only).

Connection to the controller. The dashboard does not talk to the controller directly. In control mode it starts member4_mitigation/run_experiment.py, which starts the controller, the detector and Mininet. The controller and detector write log files; the dashboard reads them. So a dashboard crash can never disturb the SDN experiment (this was tested by killing the dashboard during an attack).

C. Frontend architecture

```
  Browser  http://127.0.0.1:8050/   (index.html)
     |  GET /api/state  (1 s)   GET /api/control (2 s)   GET /api/runs, /api/health (5 s)
     |  GET /api/summary (Results view)     POST /api/control/start|stop|cleanup (control mode)
     v
  server.py  (Python http.server, localhost)
     |  state.py: read files only            control.py: run_experiment.py (sudo)
     v                                              |
  member4_mitigation/results/<run>/   <-----------+  writes
     decisions.csv  actions.csv  events.csv  port_stats.csv  sink.csv  ovs.csv
     resources.csv  meta.json  features.csv  traffic/*.csv   metrics.json
                     ^
                     |  written by
  os-ken controller (Member 1 switch + Member 2 monitor + Member 4 mitigation app)
     <== OpenFlow 1.3 ==>  Open vSwitch s1  ==  Mininet hosts h1 (victim) .. h6
  Member 3 detector (separate process): features.csv -> decisions.csv
  Member 1 generators on h2..h5 (scenario), h6 (legitimate client), sink on h1
```

Data flow from traffic to screen (one second of a naive attack):

1. Member 1's generators on h2–h5 send UDP packets to h1; h6 sends normal traffic all the time.
2. The switch counts them. Member 2's monitor asks for counters every second and writes a feature row (DataPlanePacketRate jumps from ~30 to ~300 pkt/s).
3. Member 3's live detector reads the row and writes a decision: naive_ddos, score 6.1, "rate jumped 305 in one window", rule onset_jump.
4. Member 4's app reads the decision within ~0.2 s, finds the ports whose rate is ≥ 20 pkt/s (h2–h5, not h6), installs one meter + one flow rule per port, and logs limit_sent and, when the switch confirms, limit_confirmed.
5. The sink on h1 counts packets that actually arrived per sender.
6. The dashboard reads all of these files and redraws.

Storage. No database. Each experiment has its own folder under member4_mitigation/results/; the dashboard only reads it.

D. Page-by-page explanation

The page has two views, chosen with the tabs at the top right: **Live run** and **Results**. The green pill next to them is the **server health** indicator ("server OK · read-only" or "server OK · control mode"; "server unreachable" if the server stops — the page keeps the last picture, dimmed, and recovers by itself).

D1. Live run view

1. Experiment control card (top)

Element	Meaning	Source
Scenario	normal / genuine flash crowd / naive DDoS / adaptive DDoS	fixed list = Member 1's generators
Adaptive level	subtle / moderate / aggressive (enabled only for adaptive)	Member 1's presets
Detector	combined (proposed) / threshold-only / CUSUM-only / entropy-only	Member 3 + baselines
Mitigation	on, or off (monitor only: decisions are logged, no rules)	Member 4 policy
Start experiment	POST /api/control/start → server runs run_experiment.py with these choices. About 1.5–2 minutes per run.	control.py
Stop	POST /api/control/stop → sends Ctrl+C to the runner, which stops traffic, Mininet, detector and controller cleanly; the run is marked "stopped"	control.py
Clean up Mininet	POST /api/control/cleanup → runs mn -c (only when nothing is running)	control.py
Runner output	last lines printed by the runner, including its final summary	runner log file

In read-only mode the controls are disabled and the card explains how to enable them. *Say:* "This panel starts a real Mininet experiment — the same script we use for our measurements, nothing simulated."

2. "Showing" row — status pill (LIVE · seconds, or finished/stopped) and the **run picker**: "Follow newest run" (default; jumps to a new run automatically) or any recorded run, which is then replayed in full. *Say:* "Old runs are recordings of real experiments, labelled with date, scenario and configuration."

3. Four status tiles

Tile	Meaning	Source
Detector decision (Member 3)	the latest class with icon + reason (e.g. "rate jumped 204 in one window (limit 140)")	decisions.csv
Mitigation (Member 4)	ON/OFF and how many limits were installed so far; says "monitor-only run" when mitigation is off	actions.csv, events.csv
Rate-limited hosts	chips for the hosts currently metered, and the meter rate (10 pkt/s); warns if a legitimate host is limited	events.csv + port map
Detector score	max(rate / limit, CUSUM / h); above 1.0 = abnormal. Not a probability.	decisions.csv

4. Chart "Packets each host sends into the switch" — one line per host (h2–h6, legend shows attacker/legitimate role), packets per second from the switch's **port counters**. Red shading = mitigation on. Dashed lines = offender threshold (20 pkt/s) and meter rate (10 pkt/s). Dotted vertical lines = scenario start/end. Hover shows every host's value at that second. Source: port_stats.csv (Member 4 app polls OpenFlow port statistics every second). *Say:* "This is what the attackers are trying to send. It

stays high during mitigation because the switch counts packets before the meter drops them."

5. Chart "Packets delivered to the victim h1" — packets per second that actually **arrived** at h1, per sender, measured by a small receiver (sink) on h1. Source: sink.csv. *Say:* "Here you see the meters working: the attackers drop to 10 packets per second each while h6, the legitimate user, is untouched."

Both charts have a **Data table** under them (latest and maximum value per host).

6. Decision strip — one coloured cell per 1-second window: green ✓ normal, yellow ◆ flash crowd, red ! attack, grey ? unknown. Faded red cells are "hold" windows (a pause inside an adaptive attack, during which mitigation deliberately stays on). Hover a cell for the detector's reason. Source: decisions.csv.

7. Mitigation events table — controller-app events: app started (with its settings), switch connected, limit_sent (port, observed peak rate), limit_confirmed (time the switch needed, and decision-to-rule time), remove_sent/confirmed, errors. Source: events.csv.

8. Run and resources — attacker and legitimate hosts of this run; controller and detector CPU (% of one core) and memory (MiB), sampled every second with psutil. Source: meta.json, resources.csv.

D2. Results view

All numbers come from **recorded real experiments** (33 complete runs at the time of writing); the note at the top states the count. Nothing on this page is simulated.

Section	What it shows	Source
A. Detection on identical traffic	Precision, recall, F1, FPR, FNR and false alarms on flash crowds for the four detectors, all fed the same recorded feature windows	results/offline_detection.csv (from offline_detection.py)
B. Legitimate traffic delivered	bar per scenario × configuration: delivery ratio of legitimate hosts (1.0 = nothing lost)	metrics.json of each run (/api/summary)
C. Attack traffic delivered	delivery ratio of attack hosts during the attack (lower = more blocked; "No mitigation" ≈ 1.0 is the baseline)	metrics.json
D. Timing, collateral, overhead	response time, release time, whether mitigation was lifted during the attack, legitimate/attack hosts limited, packets dropped by meters (switch's own counters), controller CPU and memory	metrics.json

Configurations (fixed colours): **Proposed** (combined detector + mitigation), **No mitigation** (combined, monitor only), **Threshold-only**, **CUSUM-only**, **Entropy-only** (each + the same mitigation).

Key results to point at (from member4_mitigation/results/):

Observation	Number
Flash crowd: legitimate delivery, proposed vs threshold-only vs CUSUM-only	1.000 vs 0.319 vs 0.371
Flash-crowd false-alarm windows (replay): combined / threshold / CUSUM	0 / 96 / 114
Naive DDoS, proposed: attack delivered pkt/s without → with mitigation	323.7 → 71.7 (PDR 0.224), response 2.3 s
Legitimate hosts limited in any attack run	0
Detection (replay, all runs): combined precision / recall / F1	0.875 / 0.656 / 0.750
Entropy-only recall	0.007 (feature is ~0 after flow setup)
Adaptive subtle	not mitigated by the proposed system (attack PDR 1.000) — the evasion we study
Controller overhead	0.4–4.1 % CPU of one core, ~46 MiB

E. Demonstration script (about 7 minutes)

Before the class (setup, ~3 minutes, not part of the demo)

```
cd ~/Documents/CN-Project
sudo mn -c                                  # clean Mininet state
ss -ltnp | grep 6653                        # must print nothing (no other controller)
sudo python3 member4_mitigation/dashboard/server.py --control
```

Expected terminal line: Member 4 dashboard: http://127.0.0.1:8050/ (CONTROL mode: can start experiments; Ctrl+C to stop). Open **http://127.0.0.1:8050/** in the browser. Check: the pill says "server OK · control mode" and the Start button is enabled. Optional rehearsal: run one naive DDoS experiment.

Minute 0–1: introduction (talk)

"Our project defends a Software Defined Network against DDoS attacks without machine learning. The difficult case is an attack that imitates a flash crowd — a real rush of users. We built four modules: traffic generation, monitoring, statistical detection, and mitigation with evaluation. This dashboard shows the complete pipeline running live on a Mininet testbed with Open vSwitch and an OpenFlow 1.3 controller."

Minute 1–2: tour on a recorded normal run

Open the **Showing** picker and choose a run ending in normal_combined_mit-on_r1. Say it is a recorded real run. Point at: green decision strip, Mitigation OFF, no limited hosts, h2/h3/h6 at about 10 pkt/s each in both charts. "Normal traffic passes without any intervention."

Minute 2–4: live naive low-rate DDoS

Set the picker back to **Follow newest run**. In the control card choose **Naive low-rate DDoS**, detector **combined (proposed)**, mitigation **on**, press **Start experiment**. Expected: message "Started…", status "Running: naive_ddos …", within ~10 s the status pill shows **LIVE**.

While it runs (~85 s), explain the timeline: controller and network start (~7 s), 10 s warm-up, then h6 (the legitimate user) starts, and 15 s later the attack starts on h2–h5.

Expected at the attack start (about 30–35 s after Start): chart 1 jumps to ~90 pkt/s per attacker; within about 2–3 s the decision tile turns red **Naive DDoS** ("rate jumped … in one window"), Mitigation **ON**, rate-limited hosts **h2 h3 h4 h5**, red shading begins; in chart 2 the attackers fall to 10 pkt/s each while h6 stays at 10. The events table shows limit sent for four ports and limit confirmed with the switch time in milliseconds. "Only the attackers are limited — to the rate of a normal user, not blocked."

About 20 s later the attack ends: decisions return to normal, Mitigation OFF, remove sent/confirmed appears. The runner output ends with the run summary.

Minute 4–5: genuine flash crowd (recorded comparison)

A live flash-crowd run takes ~1.5 minutes; to save time use recordings (labelled). Choose flash_crowd_combined_mit-on_r1: yellow **flash crowd** cells, Mitigation never ON, no host limited — "the proposed detector recognises the gradual, unsynchronised rise of real users and does not punish them." Then choose flash_crowd_threshold_mit-on_r1: the threshold-only detector called it an attack and limited h2–h4 (chips, shading, chart 2 drops). "That is exactly the false positive we wanted to avoid." (If time allows, run the flash crowd live instead with the Start button.)

Minute 5–7: live adaptive flash-crowd-mimicking DDoS + Results

Set the picker to **Follow newest run**, choose **Adaptive** with level **moderate**, detector combined, mitigation on, **Start**. The run takes ~1 min 50 s. While it warms up, open the **Results** tab and explain: section B (proposed keeps legitimate delivery at 1.0 everywhere; the single-signal detectors drop to ~0.3

on the flash crowd), section C (attack delivery cut for naive and aggressive; subtle evades), section A (precision / recall / false alarms on identical traffic; entropy-only detects almost nothing because Packet-In features are flat).

Return to **Live run** when the attack starts. Expected: bursts of ~40 pkt/s per attacker with pauses; the **first burst is labelled flash crowd** (it looks the same), at the **second burst** the class becomes **Adaptive DDoS** ("repeated bursts separated by pauses") and mitigation switches on; during the pauses faded "hold" cells keep it on. "This shows both the strength and the honest limitation: the attack is caught at its second burst, and the first burst gets through."

Stop and reset

- To stop a running experiment: **Stop** (it cleans up itself; the run is shown as "stopped").
- If Mininet complains later: **Clean up Mininet** (or sudo mn -c).
- Stop the dashboard: Ctrl+C in its terminal. A running experiment would continue and finish by itself.

Fallback if Mininet or the controller is unavailable

Start the dashboard **read-only** (no root needed):

```
python3 member4_mitigation/dashboard/server.py
```

Use the run picker to replay recorded runs (naive, flash crowd, adaptive) and the Results tab. **Say clearly that these are recordings of real experiments made on 23 October 2026, not a live run.** No simulated data is shown anywhere in the dashboard.

F. Viva preparation

1. **What problem does the project solve?** Detecting and limiting DDoS traffic in an SDN, especially low-rate attacks that imitate flash crowds, without machine learning and without blocking genuine users.
2. **Why SDN?** The controller sees the whole network and can install rules on switches instantly (OpenFlow); we measure and mitigate from one place.
3. **Why is an adaptive low-rate attack hard?** It uses realistic rates and jitter, ramps up gradually like real users, and pauses so window-based detectors reset. Per-flow rates look normal; only the aggregate pattern differs.
4. **Flash crowd vs DDoS?** Both raise the total rate. A flash crowd rises gradually and unsynchronised and persists; a naive DDoS jumps in one window (synchronised start); an adaptive DDoS shows burst–pause–burst. Member 3's rules use exactly these differences.
5. **Roles of the members?** 1: traffic generation (4 scenarios, 3 adaptive levels); 2: monitoring/features every second; 3: statistical detection (threshold, CUSUM, temporal rule); 4: mitigation with OpenFlow meters, evaluation, baselines and this dashboard.
6. **What is CUSUM?** Cumulative sum: $S = \max(0, S + x - \mu - k)$; alarm when $S > h$. It accumulates small persistent excesses, so it detects slow, sustained increases a fixed threshold misses. Member 3 caps S at 2h so the alarm clears soon after the attack.
7. **What is entropy here, and why is it not used for decisions?** Shannon entropy of source IPs in Packet-In messages. Member 1's controller installs permanent flows, so after setup almost no Packet-Ins occur; the feature is ~0. We implemented and measured an entropy-only baseline: recall 0.007.
8. **Threshold detection and its limits?** Alarm if the rate exceeds 3× the normal baseline (60 pkt/s). Simple and fast, but it flags flash crowds (96 false windows) and misses attacks below the limit (subtle level).
9. **How does mitigation work?** On an attack decision the app reads per-port packet counters; ports that sent ≥ 20 pkt/s in the last 10 s get a meter that drops traffic above 10 pkt/s. Removed after the first clean window (Member 3's hold covers adaptive pauses).

10. **What OpenFlow rules exactly?** Per port P: a MeterMod (drop band, 10 pkt/s, burst 10) and a FlowMod in_port=P → meter P → output:NORMAL, priority 100, cookie 0x4d34; a Barrier confirms installation. Release deletes flows by cookie and the meters.
11. **Why rate-limit instead of block?** A misjudged legitimate host keeps working at a normal rate; the attacker loses most of its traffic.
12. **Did you verify the switch supports meters?** Yes: ovs-ofctl meter-features reports max_meter 200000 and drop bands; during runs we also read the switch's own flow/meter list (ovs.csv) — it always matched the app's log and returned to 0 afterwards.
13. **False positive / false negative?** FP: a normal or flash-crowd window called an attack (punishes users). FN: an attack window missed (attack traffic gets through). We report FPR and FNR per detector.
14. **Which metrics and how measured?** Precision, recall, F1, FPR, FNR from per-window decisions vs ground truth from the generators' send logs; detection latency; response time (attack start → switch confirms meter); packet delivery ratio and throughput (packets received at h1 ÷ sent); flow-table size and Packet-In from Member 2; CPU/memory with psutil.
15. **Baseline comparison results?** On identical recorded traffic: combined precision 0.875 with 0 flash-crowd false alarms; threshold 0.832 (96); CUSUM 0.690 (114) but highest recall 0.912; entropy recall 0.007. With mitigation, single-signal detectors cut legitimate flash-crowd delivery to 0.32–0.37; the proposed system kept 1.0.
16. **How do frontend and backend communicate?** The page polls JSON endpoints of a Python standard-library HTTP server every 1–5 s; control actions are POST requests accepted only from the page itself. The server reads the experiment's files; in control mode it launches the runner.
17. **What are the limitations?** One traffic feature drives detection; adaptive attacks are caught only at the second burst; the subtle level evades; attribution assumes one host per port and no spoofing; no controller-side FlowMod throttling; one machine and few repeats.
18. **Future work?** Idle timeouts so Packet-In features (entropy, jitter) become usable; per-flow attribution; adaptive thresholds; repeated runs with confidence intervals; larger multi-switch topologies.

G. Troubleshooting

Problem	Fix
Browser: "This site can't be reached"	Dashboard server not running: start it; check the terminal for errors
Pill says "server unreachable"	Server stopped; restart it — the page reconnects by itself
Start button disabled	Server is read-only: restart with sudo python3 member4_mitigation/dashboard/server.py --control
--control launches Mininet experiments and needs root	Use sudo
Address already in use (dashboard)	Another dashboard on 8050: stop it or add --port 8051
Start → "an experiment is already running"	Wait, or press Stop
Runner output: port 6653 is already in use	Another controller is running: ss -ltnp | grep 6653, stop it
Runner output: Mininet / s1 / interface errors	Press Clean up Mininet (or sudo mn -c) and start again
Charts empty for a few seconds after Start	Normal: controller start + 10 s warm-up before traffic
HubThread … has no attribute 'kill' in a controller log	os-ken's normal shutdown message (harmless)
Permission denied … __pycache__	Root-owned cache from an earlier run: sudo rm -rf trafficgen/__pycache__
No CORS issues	The page and the API are served by the same server (same origin); no CORS configuration is needed

---

H. Limitations and future work

Limitations (real, measured): detection depends on DataPlanePacketRate only; adaptive attacks are recognised at the second burst (response ≈ 18 s for moderate); the subtle level is not mitigated by the proposed system; single-signal baselines lift mitigation during attack pauses; offender attribution assumes one host per switch port and no address spoofing; controller-side FlowMod throttling is not implemented; results come from one machine, one switch and six hosts with mostly one run per configuration.

Future work: flow idle timeouts in the controller (so entropy and jitter become informative), per-flow attribution, adaptive baselines, repeated runs with confidence intervals, multi-switch topologies, and an event stream instead of polling for larger deployments.

I. Quick revision sheet

Pipeline: Member 1 traffic (Mininet h2–h6 → h1) → OVS s1 → Member 2 features every 1 s → Member 3 decision every 1 s → Member 4 meters on offending ports → logs → evaluator → dashboard.

Classes: normal · flash_crowd (legitimate) · naive_ddos · adaptive_ddos (+ unknown).

Detector rules: threshold rate > 3×20 = 60 pkt/s; CUSUM k = 5, h = 40, capped at 80; naive = jump ≥ 140 pkt/s in one window; adaptive = ≥ 2 bursts with a 3–15 s pause; hold 6 windows.

Mitigation: offender ≥ 20 pkt/s in last 10 s → meter 10 pkt/s (burst 10), flow in_port → meter → NORMAL, priority 100, cookie 0x4d34; release after 1 clean window.

Key numbers: flash crowd legit delivery 1.000 (proposed) vs 0.319 / 0.371; naive attack 323.7 → 71.7 pkt/s, response 2.3 s; 0 legit hosts limited; combined precision 0.875, recall 0.656; entropy recall 0.007; CPU ≤ 4.1 %.

Commands:

```
sudo python3 member4_mitigation/dashboard/server.py --control   # demo, then open http://127.0.0.1:8050/
python3 member4_mitigation/dashboard/server.py                  # read-only fallback
sudo python3 member4_mitigation/run_experiment.py --scenario naive_ddos
bash run_tests.sh                                               # all automated checks
bash member4_mitigation/verify_testbed.sh                       # meters + controller + pingall
sudo mn -c                                                      # reset Mininet
```

Demo order: setup → intro → recorded normal → live naive → recorded flash crowd (proposed vs threshold) → live adaptive + Results tab → Stop / reset.